Rapport de test — Qwen3 14B sur l'agent de revue des PR

Nov 9, 2026 · @oussama

Conclusion

Qwen3 14B, appelé par l'API LLM Calipso, fait nettement mieux que qwen2.5-coder 7b : les 8 remarques qu'il a produites sont toutes pertinentes, il classe l'injection SQL en « bloquant » et il répond en 17 à 40 secondes par PR avec /chat, contre 53 à 215 secondes.

L'endpoint `/chat` est à retenir : aussi précis que `/think`, il n'a eu aucune erreur serveur, alors que `/think` a échoué 2 fois sur 3 sur la même PR. Le contrôle d'accès manquant n'est détecté qu'une fois sur trois : c'est le point à traiter en priorité, en donnant plus de contexte au modèle.

L'échantillon est très petit (4 PR de test, 3 passages par endpoint) : c'est un signal encourageant, pas encore le benchmark de 30 à 50 PR demandé.

Configuration testée

Le test compare deux endpoints du même modèle avec la référence de la semaine dernière, sur le même banc de test.

Élément	Valeur
Modèle	Qwen3 14B quantifié AWQ (`qwen3-14b`), via l'API LLM Calipso v1.1
Endpoints	`/api/qwen3/think` (raisonnement explicite) et `/api/qwen3/chat` (format OpenAI, raisonne aussi)
Paramètres	température 0,1 ; `max_tokens` 6 144 ; `max_thinking_tokens` 3 072 (think uniquement)
Format de sortie	schéma JSON donné dans le prompt, réponse vérifiée par l'agent (l'API ne force pas le JSON)
Référence	qwen2.5-coder 7b via Ollama, sur CPU (8 vCPU, 8 Go), testé le 5 novembre
Banc	vrai agent, Redis et worker ; faux GitHub et boîte mail de test (Mailpit)
Charge	une analyse à la fois, aucune charge simultanée

Côté agent, un client dédié à cette API a été ajouté ; on l'active avec `LLM_PROVIDER=qwen3api` et `QWEN3_ENDPOINT=chat`. La clé d'API est stockée uniquement dans le fichier `.env` local, exclu de Git. L'hôte du serveur n'est pas dans la liste par défaut des hôtes autorisés : il a été ajouté pour ces passages seulement.

Scénarios

Quatre PR fictives couvrent chacune un chemin de l'agent ; aucun code réel de Calipso n'a été envoyé au modèle.

PR	Contenu	Attendu d'un bon relecteur
#101	Contrôleur Symfony de suppression de contrat, avec une clé AWS en dur	Injection SQL ligne 20 (bloquant), `dump()` oublié ligne 21, route sans contrôle d'accès ligne 16-17 ; la clé est traitée par le détecteur de secrets, pas par le modèle
#102	Service de calcul de prime propre, avec son test	Aucune remarque

PR	Contenu	Attendu d'un bon relecteur
#103	Fichier de 2 000 lignes et le même service, sans test	Fichier signalé trop volumineux ; l'absence de test est une remarque légitime selon nos règles
#104	Webhook reçu pour un ancien commit	Ignoré, rien publié

Les contrôles indépendants du modèle (statut de la PR, email d'alerte, secret jamais divulgué, doublons) passent sur tous les passages.

Résultats

Les deux endpoints Qwen3 trouvent systématiquement l'injection SQL et le `dump()`, sans fausse alerte ; seul `/chat` est stable.

Critère	qwen2.5-coder 7b (CPU)	Qwen3 14B `/think`	Qwen3 14B `/chat`
Injection SQL (#101)	trouvée, classée « majeur »	3/3, classée « bloquant »	3/3, classée « bloquant »
`dump()` oublié (#101)	trouvé	3/3	3/3
Route sans contrôle d'accès (#101)	manquée	1/3	1/3
Fausses alertes sur la PR propre (#102)	1 remarque mineure inutile	aucune	aucune (3/3)
Remarques pertinentes	environ 2 sur 5	8 sur 8	8 sur 8
Erreurs serveur	—	2 analyses sur 7 (502, connexion coupée)	aucune sur 7
Durée par PR	53 à 215 s	19 à 76 s	17 à 40 s

Détail des passages (#103 n'a été jouée qu'au premier passage : 1 remarque légitime « test manquant » sur les deux endpoints, en 19 s environ) :

Passage	#101 : défauts trouvés	#101	#102 : résultat	#102
`/think` 1	SQL, `dump()`	20,2 s	erreur 502 → analyse incomplète	76,2 s

Passage	#101 : défauts trouvés	#101	#102 : résultat	#102
`/think` 2	SQL, `dump()`, contrôle d'accès	22,7 s	aucune remarque	26,3 s
`/think` 3	SQL, `dump()` (classé « mineur »)	20,2 s	connexion coupée → analyse incomplète	45,4 s
`/chat` 1	SQL, `dump()`, contrôle d'accès	27,8 s	aucune remarque	40,3 s
`/chat` 2	SQL, `dump()`	17,5 s	aucune remarque	31,4 s
`/chat` 3	SQL, `dump()`	21,0 s	aucune remarque	39,4 s

Quand `/think` a échoué, l'agent a réagi comme prévu : la PR est passée en « à vérifier » avec la mention « revue incomplète », sans feu vert. Les durées incluent les tentatives de reprise.

Limites et risques

Les résultats sont prometteurs mais ne suffisent pas à dimensionner ni à valider le modèle.

- **Échantillon très réduit** : 4 PR fictives, 3 passages par endpoint. Les « 8 sur 8 » ne valent pas mesure de pertinence ; il faut le benchmark de 30 à 50 PR avec des défauts étiquetés par un relecteur.
- **Résultats variables** d'un passage à l'autre : le contrôle d'accès n'est trouvé qu'une fois sur trois et la gravité du `dump()` varie (majeur ou mineur). Le modèle ne voit que le diff, pas la configuration de sécurité ni les routes : cause probable à confirmer.
- **Stabilité de `/think`** : 2 échecs sur 7 appels (erreur 502, connexion coupée), tous deux sur la même PR. Cause côté serveur non identifiée (délai du proxy ou raisonnement trop long, à vérifier avec l'équipe infra).
- **Pas de mesure de charge** : une seule analyse à la fois ; ni le p95 sous charge, ni la consommation mémoire du serveur n'ont été mesurés.
- **Transport non chiffré** : l'API est exposée en HTTP sur une adresse IP publique ; la clé et le code circulent en clair sur Internet. Sans conséquence avec des PR fictives, bloquant pour du code réel.
- **Validation préalable** : si ce serveur n'est pas considéré comme interne, son usage pour du code réel doit être validé au préalable, comme demandé par Hicham.

Recommandations

Retenir Qwen3 14B via `/chat` comme nouvelle référence du benchmark, à la place de qwen2.5-coder 7b, et le comparer aux deux modèles proposés par Hicham (Qwen3-Coder-Next, Qwen3-Coder-30B-A3B-Instruct).

- [ ] Faire passer l'API en HTTPS ou sur un réseau privé avant tout essai sur du code réel (équipe infra)
- [ ] Confirmer que ce serveur compte comme hébergement interne, ou obtenir la validation préalable
- [ ] Analyser les erreurs de `/think` avec l'équipe infra (journaux du serveur, délai du proxy)
- [ ] Ajouter au contexte du modèle les routes, la configuration de sécurité et les fonctions liées, puis rejouer #101 comme premier cas de non-régression sur le contrôle d'accès
- [ ] Construire le jeu de 30 à 50 PR étiquetées (changements sains, défauts connus, cas de droits d'accès) avec 1 à 2 relecteurs
- [ ] Mesurer sous charge : p95 attente comprise, analyses simultanées, mémoire du serveur